

Reviewer Pack — Lie-Stabilizer Dimension of Clause Cubic Bounds DPLL Leaves

Entry 77253411c65d · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-04-28 12:19:23 UTC

Lie-Stabilizer Dimension of Clause Cubic Bounds DPLL Leaves

Entry ID: 77253411c65d **Verdict:** INCONCLUSIVE **Recorded:** 2026-04-28 12:19:23 UTC

1. Conjecture

Field A (mathematical branch): Geometric Complexity Theory: dimension of the stabilizer Lie algebra $\mathfrak{g}_p = \{A \in \text{Mat}_n(Q) : \sum_i (A \cdot x)_i \partial_i p(x) \equiv 0 \text{ as a polynomial}\}$ of a homogeneous polynomial p , computed by Gaussian elimination on the linear system extracted from the cubic-monomial coefficients; the canonical first-order Mulmuley-Sohoni / Bürgisser-Ikenmeyer obstruction used to separate $\det_n$ from perm_n via stabilizer dimension ($2n^2-1$ vs. $\sim 2n-2$). Distinct from apolar/-catalecticant rank and from Hessian rank. **Field B** (complexity object): Lex-DPLL search-tree leaf count $L_{\text{DPLL}}(F)$ on small UNSAT 3-CNF formulas at clause density α , equivalently tree-resolution leaf count up to constants — the canonical proof-complexity proxy for refutation hardness.

Statement:

Define $p_F \in Q[x_1, \dots, x_n]$ by $p_F(x) = \sum \{\text{clauses } C\} \varepsilon_C \cdot x_{i_1} x_{i_2} x_{i_3}$, where $\varepsilon_C = (-1)^{\#\text{negated literals in } C}$ and (i_1, i_2, i_3) are the indices of C 's variables; let $\mathfrak{g}_F$ be its GL_n stabilizer Lie algebra and $\dim_Q(\mathfrak{g}_F)$ its dimension. Conjecture: for every UNSAT 3-CNF F on $n \geq 8$ variables, $\log_2 L_{\text{DPLL}}(F) + \dim_Q(\mathfrak{g}_F) \leq n + 2 \cdot \lceil \log_2 n \rceil$. A single UNSAT F violating this inequality (with positive margin) refutes it.

Rationale (proposer's reasoning):

GCT identifies high stabilizer dimension with maximally structured polynomials (the determinant's huge symmetry is exactly what trivializes its computation), and the orbit-closure containment program reads lower bounds off symmetry-budget mismatches. Translated to refutations, an UNSAT formula whose clause cubic carries large Lie symmetry should expose unit-propagation/decomposition structure, while symmetry-poor cubics correspond to the 'rigid' regime where DPLL is forced to branch combinatorially. Cubic clause polynomials are the cheapest GCT-style invariant capturing 3-SAT, and their stabilizer Lie algebras have not, to my knowledge, been linked to proof complexity.

Taxonomy category: GCT (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 4d70594aff1b1ddf

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

Across 1000 random UNSAT 3-CNFs (200 per $n \in \{8, 10, 12, 14, 16\}$, $\alpha = 4.5$), compute $\Delta(F) = \log_2 L_DPLL(F) + \dim_Q(g_F) - (n + 2 \lceil \log_2 n \rceil)$. SUPPORTED iff $\max \Delta(F) \leq 0$ over all 1000 instances. FALSIFIED iff any single F gives $\Delta(F) > 1$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.92	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.82	SAFE	SAFE
ALGEBRIZATION	SAFE	0.82	SAFE	SAFE
KARP_LIPTON	SAFE	0.86	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - stabilizer Lie algebra cubic form DPLL resolution complexity - geometric complexity theory 3-CNF polynomial obstruction proof complexity - Mulmuley Sohoni stabilizer dimension SAT clause cubic polynomial

Top relevant hits considered: - [<http://arxiv.org/abs/1212.5380v2>] On properties of principal elements of Frobenius Lie algebras - [<http://arxiv.org/abs/2411.11095v3>] Invariant theory and coefficient algebras of Lie algebras - [<http://arxiv.org/abs/1401.3795v2>] Relationship between Nichols braided Lie algebras and Nichols algebras - [<http://arxiv.org/abs/1611.00827v2>] Geometric complexity theory and matrix powering - [<http://arxiv.org/abs/0911.3482v5>] Complexity of Networks (reprise) - [<http://arxiv.org/abs/nlin/0101006v1>] On Complexity and Emergence - [<http://arxiv.org/abs/1908.01624v1>] Learned Clause Minimization in Parallel SAT Solvers - [<http://arxiv.org/abs/2207.13577v2>] Scalable Proof Producing Multi-Threaded SAT Solving with Gimsatul through Sharing instead of Copying Clauses

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def binomial(n, k):
    if k > n:
        return 0
    if k == 0 or k == n:
        return 1
    k = min(k, n - k)
    c = Fraction(1, 1)
    for i in range(1, k + 1):
        c *= (n - i + 1) / i
    return c

def gaussian_elimination(M):
```

```

m, n = len(M), len(M[0])
rank = 0
for j in range(n):
    pivot_row = None
    for i in range(rank, m):
        if M[i][j] != 0:
            pivot_row = i
            break
    if pivot_row is None:
        continue
    M[pivot_row], M[rank] = M[rank], M[pivot_row]
    rank += 1
    for i in range(rank, m):
        factor = -M[i][j] / M[rank-1][j]
        for k in range(j, n):
            M[i][k] += factor * M[rank-1][k]
return rank

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n_values = [8, 10, 12, 14, 16]
    results = []

    for n in n_values:
        L_DPLL = 0
        for _ in range(200):
            # Generate a random UNSAT 3-CNF formula with  $\alpha=4.5$ 
            clauses = []
            for i in range(n * (n - 1) // 2):
                clause = [random.choice([-1, 1]) * random.randint(1, n) for _ in range(3)]
                if len(set(clause)) == 3:
                    clauses.append(clause)
            F = clauses

            # Compute  $p_F$ 
            p_F_coeffs = [[0] * (n + 2) for _ in range(n + 2)]
            for clause in F:
                i1, i2, i3 = sorted(abs(lit) - 1 for lit in clause)
                sign = (-1) ** sum(lit < 0 for lit in clause)
                p_F_coeffs[i1][i2] += sign
                p_F_coeffs[i1][i3] += sign
                p_F_coeffs[i2][i3] += sign

            # Construct the constraint matrix  $M$ 
            M = [[0] * (n**2) for _ in range(binomial(n+2, 3))]
            for i in range(1, n + 2):
                for j in range(i + 1, n + 2):
                    for k in range(j + 1, n + 2):
                        index = binomial(i-1, 2) * (n - i + 1) // 2 + binomial(j-i, 2)
                        M[index][i*n+j-1] += p_F_coeffs[i][j]
                        M[index][i*n+k-1] += p_F_coeffs[i][k]
                        M[index][(j-1)*n+k-1] += p_F_coeffs[j][k]

            # Compute  $\dim_Q(g_F)$ 
            rank = gaussian_elimination(M)
            g_F_dim = n**2 - rank

```

```

# Run lex-DPLL to count L_DPLL(F)
def dpll(clauses, assignment):
    if not clauses:
        return 1
    unit_clauses = [c for c in clauses if len(c) == 1]
    if unit_clauses:
        lit = unit_clauses[0][0]
        new_assignment = assignment.copy()
        new_assignment[lit] = True
        if all(assignment[lit] == (lit > 0) or assignment[-lit] == (lit < 0) for lit in unit_clauses):
            return dpll([c for c in clauses if lit not in c], new_assignment)
        else:
            return 0
    literals = set(lit for clause in clauses for lit in clause)
    for lit in literals:
        new_assignment = assignment.copy()
        new_assignment[lit] = True
        if all(assignment[lit] == (lit > 0) or assignment[-lit] == (lit < 0) for lit in literals):
            count = dpll([c for c in clauses if lit not in c], new_assignment)
            if count > 0:
                return count
        new_assignment[lit] = False
        if all(assignment[lit] == (lit > 0) or assignment[-lit] == (lit < 0) for lit in literals):
            count = dpll([c for c in clauses if lit not in c], new_assignment)
            if count > 0:
                return count
    return 0

L_DPLL += dpll(F, {})

results.append({
    "n": n,
    "L_DPLL": L_DPLL,
    "g_F_dim": g_F_dim
})

max_delta = max(math.log2(result["L_DPLL"]) + result["g_F_dim"] - (result["n"] + 2 * math.ceil(n/2)))
conjecture_holds = max_delta <= 0
counterexample = "" if conjecture_holds else "max Δ(F) > 1"

return {
    "metric_name": "Δ(F)",
    "metric_value": max_delta,
    "instances_tested": len(results),
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]] or [11, 23, 37, 53, 71]

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")

```

```

# Compute mean/std of metric_value and fraction of seeds where conjecture_holds
total_metric_value = sum(result["metric_value"] for result in results)
mean_metric_value = total_metric_value / len(results)

squared_diff_sum = sum((result["metric_value"] - mean_metric_value) ** 2 for result in results)
std_metric_value = math.sqrt(squared_diff_sum / len(results))

support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

if all(result["conjecture_holds"] for result in results):
    print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction={support_fraction}")
elif sum(1 for result in results if not result["conjecture_holds"]) / len(results) >= 0.8:
    print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction={support_fraction}")
else:
    first_failing_seed = next(seed for seed, result in enumerate(results) if not result["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample=\"max Δ(F) > 1\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_92d0288f.py", line 139, in <module>
    result = run_trial(seed)
              ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_92d0288f.py", line 74, in run_trial
    M = [[0] * (n**2) for _ in range(binomial(n+2, 3))]
                                   ^^^^^^^^^^^^^^^^^^^^^
TypeError: 'float' object cannot be interpreted as an integer

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed with a `TypeError` (`binomial` returned a float passed to `range`) before any trials were run, so no $\Delta(F)$ values were produced and neither the support nor falsification condition can be evaluated. | next: Fix the `binomial` call by casting to `int` (or use `math.comb`) and rerun the 1000-instance sweep to obtain actual $\Delta(F)$ statistics.

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	claude_max	opus	0	0	161118
2	preregistration	claude_max	opus	0	0	5143
3	novelty	claude_max	opus	0	0	3261
4	novelty	claude_max	opus	0	0	9458
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	19272
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	18421
7	judge	claude_max	opus	0	0	5554

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 222227 ms total latency. Provider mix: {'claude_max': 5, 'ollama_remote': 2}

(full prompt+response transcripts available in `research/audit/77253411c65d.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/77253411c65d.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/77253411c65d.tar.gz` (if generated)